CIS 3250: Software Design lll
Dr. John Akinyemi
October 15, 2026
Due: Oct 28, presentation due Nov 3

Team: Almanac

Group Members:

- Addison Broschell
- Arianna Wallace
- Harveen Harveen
- Leo Goebel
- Aithan McIntyre

1. Design patterns and degenerate patterns (Arianna and Leo)
2. Decay signature comparison : Human vs agent (aithan)
3. Change plan: safe change and impact (harveen and addison)
4. Agent Appendix

1. Design patterns and degenerate patterns

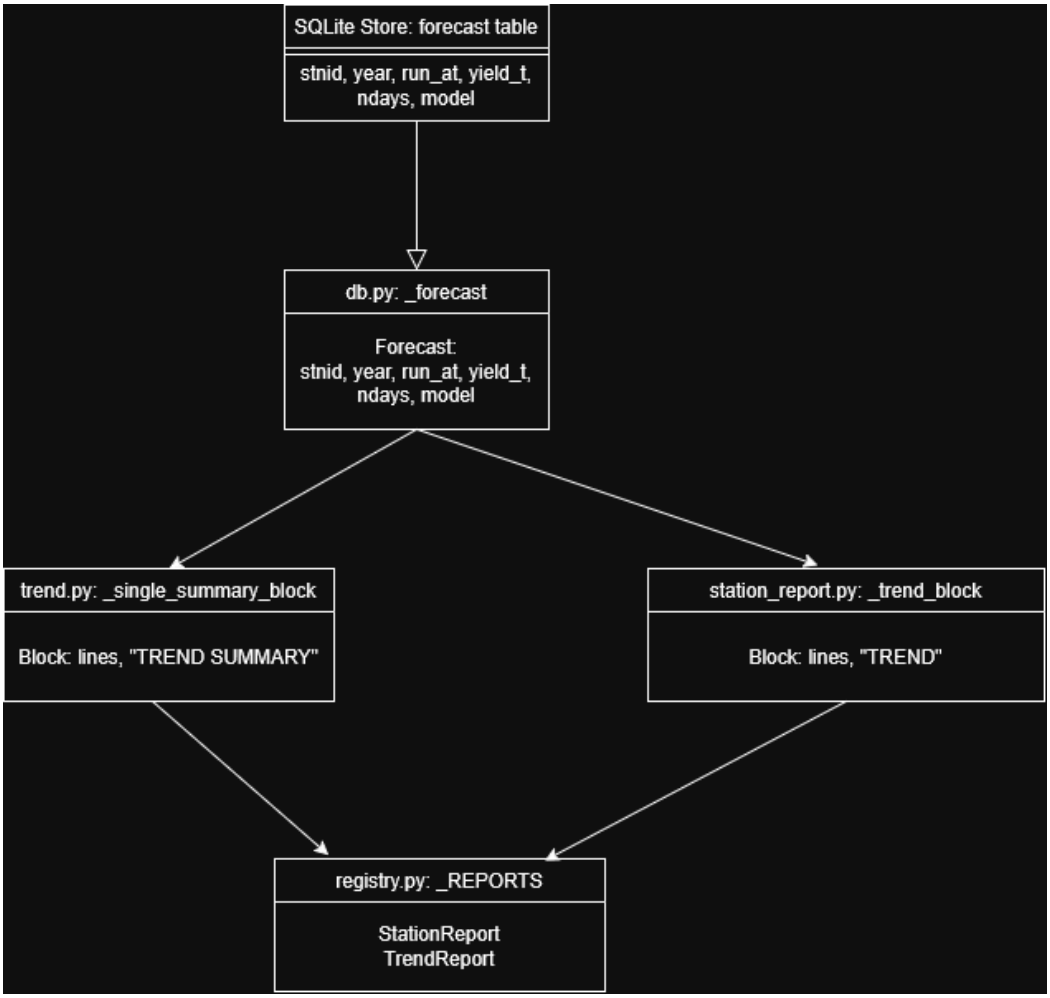
One Genuine Pattern - Move Behavior Close To Data

Files: db.py, station_report.py, trend.py.

Codebase Lines: Forecast(lines 50-56 in db.py), trend_block(lines 127-141 in station_report.py), single_summary_block(lines 192-206 in trend.py)

Kept or Removed: We choose to keep this pattern behavior as the multiple manipulators of forecast (one for each client) strengthens encapsulation having behaviour within the class containing the data it operates on. Specifically within station_report.py, method _trend_block and trend.py, method _single_summary_block they repeat the same access and code for their clients instead of an indirect client being called.

Diagram:



One Degenerate Pattern - Eliminate Navigation Code

Files: emitters.py, columns.py

Methods: emit_csv, emit_json, ColumnSet (class), Block(class)

Codebase Lines: emit_csv(lines 143-166 in emitters.py), emit_json(lines 183-240 in emitters.py)

Kept or Removed: We decided this pattern should be removed. The functions (emit_csv,emit_json) avoids using the column file, which contains helper functions and ways to access the fields within the data, bypassing it and instead using manual navigation (for example: doc (ReportDocument)⟶block (Block)⟶cs (ColumnSet)⟶cs.columns (List[Column])⟶c.title,c.key,c.unit,c.kind). This is problematic because if a small change is made to a variable name or to the order of which the data is being parsed, it would cause both functions (emit_csv, emit_json) to fail.

2. Decay-signature comparison - Human vs Agent
Three decay signatures, with file and line evidence from both meridian and jq
The human versus agent comparison and a brief maintenance conclusion
Agent appendix entry

1. **Duplicated logic**
   Meridian - reporting/reporting/reports/station_report.py : 127-141 and
   reporting/reporting/reports/trend.py : 192-205

There are two reports doing the same calculations:

- station report calculates average yield, best year, worst year and the change from the first to last year
- Trend report repeats those same calculations, then adds extra information

Instead of both reports calling one shared function, each has its own copy of the calculation code.

This is an issue because if you fix the station report but forget to fix the trend report, the two reports could give different averages for the same data. The Git history shows the station report existed first. When the trend report was added later in commit df130ef in August 2026, it repeated the existing logic.

Jq - src/jv.c:337-355 and src/jv.c:376-394

Two parts of the pthread_key_create() function repeat similar steps for setting up memory:

- One Creates the first storage block for thread information
- The other creates an additional block when more space is needed

Both sections allocate memory, check whether the allocation succeeded, and initialize the storage entries.

This creates a maintenance problem because changes to these shared steps may need to be made in both places. Updating one section and forgetting the other could cause inconsistent behaviour. The comment XXX DRY at line 377 acknowledges the repetition. DRY means "Don't Repeat Yourself"

The Git history shows that this code was added in commit 34182cca in 2021 as part of a windows support fix.

Human versus agent comparison

In jq, the duplicated logic was introduced while adding Windows support, and the developer acknowledged it with the XXX DRY comment. This shows how human maintenance can introduce repetition while solving a specific practical problem.

In Meridian, the newer trend report repeats calculations already used by the station report. The new report looks sensible by itself, but it adds another separate implementation that must stay consistent with the existing system. This illustrates the agent-era problem of producing a plausible feature without fully integrating it with the surrounding code.

Both create the same maintenance risk: a developer might update one copy and miss the other. The difference is the documented context in which the duplication appeared.

2. Comment-versus-code drift

Meridian - reporting/reporting/README.md:29-36, 48

The reporting README does not match all the features available in the code:

- It lists five reports but leaves out the trend report
- It lists text and CSV output but leaves out JSON

The trend report is included in

- reporting/reporting/reports/registry.py : 29-35, and JSON is supported in …

  reporting/ reporting/config.py : 26 - 30

This creates a maintenance problem because someone relying on the README could misunderstand what the system supports. The code has newer features, but the documentation still describes the older functionality.

Git history shows that commit df130ef added the trend report and JSON export in August 2026 without updating the README>.

jq – src/jv.h:45–48 and src/jv.c:1892–1897

- The comment in jv.h says all jv_* functions except jv_copy decrease their input's reference count, which helps manage memory. However, jv_get_refcnt() only reads the count. It does not decrease it. The comment therefore does not accurately describe every function.

- A developer relying on this comment could misunderstand how to manage memory. Git history shows that the comment dates to 2012 and remained unchanged while the function's implementation evolved.

Human versus agent comparison

In jq, an incomplete comment survived years of human maintenance. In Meridian, newer agent-era features were added without updating the existing documentation. Both leave maintainers with incomplete descriptions, but jq shows longstanding documentation debt, while Meridian shows a gap between new features and the documentation surrounding them.

3. Incomplete shared abstractions

Meridian – reporting/reporting/reports/selection.py:3–20, 55–69

The selection module says that choosing the current forecast has been centralized into one shared rule. However, the reports still use different methods:

- reporting/reporting/reports/trend.py:137 uses the shared current_forecast() helper.
- reporting/reporting/reports/station_report.py:106 uses a separate database method.

These methods do not follow identical rules. The shared helper chooses the newest crop year when two forecasts have the same timestamp. The database query in reporting/reporting/db.py:208–214 does not specify that tie-breaking rule.

This means the reports can disagree about which forecast is current. Updating the shared helper would also leave the separate database logic unchanged.

Git history shows that commit 3159c9a, titled “centralise current-run selection,” added only selection.py. It did not replace the station report’s separate selection path. The new helper looks reasonable individually, but the claimed centralization is incomplete.

jq - src/util.c:261 - 265 and src/itil.c:284 - 288

jq reports input errors through two different routes:

- If opening a file fails, it uses a configurable error callback – a function the caller can provide to handle errors.
- If reading an opened file fails, it prints directly to standard error, bypassing that callback.

The comment at line 263 - 264 explains that the callback only handles opening-related problems. Therefore, a caller relying on the callback cannot collect every input error through the same mechanism.

Git history shows that commit 5e0db6dd, titled “partial handling of input errors” , introduced the separate reading - error path in 2015.

Human versus agent comparison

jqs history explicitly describes a partial solution: human maintenance added error handling without fully extending the existing callback. Meridian presents its newer helper as a centralized solution, although another selection path remains separate. Both leave gaps in a shared mechanism but meridian particularly illustrates code that looks complete individually without being consistently integrated into the system.

3. Change plan for your assigned subsystem - Impact

From looking in the ISSUES.md, we found three open tickets relating to our subsystem: MRD-238, MRD-250, and MRD-251. Our team is proposing the changes from tickets MRD-238 and MRD-250. These changes were chosen because these two tickets address the same main problem: the forecast runs are selected by comparing their timestamps as text, which can lead to an older forecast being selected if/when the timestamps use different formats.
Both of these tickets address the same problem, but they affect different parts of our subsystem. Because of this, we can treat them as one large issue with two separate code changes.

Change 1: MRD-238

Location: reporting/reporting/db.py : lines 195-220.

Modules and Interfaces: station_report.py calls latest_forecast_for_station(stn) on line 106 to retrieve the latest forecast for a selected station. The base.py module also uses latest_run_at() to retrieve the timestamp that is displayed in the report banner. Both of them use SQL MAX(run_at) to select the newest timestamp.

Risk - What would break: In the MRD-238 ticket, the db.py compares timestamps stored as text and assumes the analytics job writes them in a consistent format. If the timestamps use different formats, MAX(run_at) may select the older forecast instead of the newest one. This could affect the data displayed in station reports. The latest_run_at() method uses the same text-ordering assumption, so the timestamp displayed in the report banner could also be incorrect.

How to know: Check the timestamp formats in the available data and test both database methods with consistently formatted timestamps. If mixed formats are valid inputs, add tests to confirm that the chronologically newest timestamp is selected. Rerun the existing tests in test_reports.py and compare the affected station report and banner outputs.

Dependencies: station_report.py depends on latest_forecast_for_station() to retrieve the station's forecast, while base.py uses latest_run_at() for the report banner. Both methods need to be

considered when addressing the timestamp comparison issue. Existing report tests should still pass, with additional tests covering the risk.

Defensible order of work:

1. Check the timestamp formats used by the analytics job and the available data.
2. Determine how the database methods should compare timestamps chronologically and update the selection logic.
3. Test latest_forecast_for_station() and latest_run_at() with the supported timestamp formats.
4. Rerun the existing report tests and compare station report and banner outputs.

Change 2: MRD-250
Location: reporting/reporting/reports/selection.py : lines 42-101

Modules and Interfaces: _recency_key() returns a timestamp string, which current_forecast() uses when selecting the most recent forecast. The trend.py module calls selection.current_forecast(forecasts) on lines 137 and 270 for single-station and multi-station trend reports. The related functions current_run_at(), order_by_recency(), and current_by_station() also rely on the shared selection logic.

Risk - What would break: _recency_key() compares timestamps as text, assuming their text order matches their chronological order. If timestamps use different formats, current_forecast() may select an older forecast. This could affect both single-station and multi-station trend reports. The same issue could affect the ordering of forecasts and the current timestamp returned by the other selection functions. The crop-year tie-breaker makes selection deterministic when timestamp keys are equal, but it does not correct an incorrect timestamp ordering.

How to know: Test current_forecast() with consistently formatted timestamps and, if mixed formats are valid inputs, with mixed-format timestamps. Test current_by_station(), current_run_at(), and order_by_recency() to check that they use the correct selection and ordering. Rerun the existing tests in test_trend.py, add tests for the identified risk, and compare the affected trend report outputs.

Dependencies: The trend reports depend on current_forecast() to select the current forecast. The related selection functions use _recency_key() as their comparison key, so changes to this helper could affect several report behaviours. The existing selection tests and both single-station and multi-station trend reports need to be checked after any changes.

Defensible order of work:

1. Check the timestamp formats used by the analytics job and the available data.
2. Determine how _recency_key() should compare timestamps chronologically and update the shared selection logic.
3. Test current_forecast() and the related selection functions using the supported timestamp formats.
4. Rerun the existing trend tests and compare single-station and multi-station trend report outputs.

3. Change plan for your assigned subsystem - Safe Change

Capturing Current Behaviour

Before making any changes to the Reporting subsystem, we will establish a baseline by building and running Meridian using make all and ./run.sh , as required by the assignment. We will save the relevant outputs so that we can compare the system's behaviour before and after a future change.

During the initial run, the Station Roster dashboard displayed six monitoring stations: BRANTFORD, CAMBRIDGE, GUELPH, KITCHNER, MILTON and WATERLOO. It showed 1,044 station-days, coverage for 2025, and an average record length of 174 days of records, and all six stations displayed a neutral yield-trend indicator (→).

We will save this output as our initial system-level baseline. However, because our proposed changes (MRD 238 and MRD 250) concern forecast selection in the reporting subsystem, we will also capture the output of the affected station and Trend reports using the default database. This will allow us to compare the reports before and after a future change and verify that they select the correct current forecast and display the correct timestamp in the report banner. The dashboard output alone cannot confirm that the reporting subsystem is selecting the correct forecast.

Characterization Tests

Before implementing the change, we will run characterization tests to record the existing behaviour and protect against regressions.

- **MRD-238 - Latest forecast selection**: latest_forecast_for_station() using forecasts with consistently formatted and mixed format timestamps. Verify that it selects chronologically the newest forecast for the station. This protects the station report from displaying outdated forecast information.
- **MRD-238 - Report banner timestamp:** Test latest_run_at() with different supported timestamp formats and verify that the report banner displays the correct latest run timestamp.

- **MRD-250 - forecast recency:** Test current_forecast() with timestamps in different supported formats that represent different times. Verify that it selects the chronologically newest forecast rather than relying on text ordering.
- **MRD-250 - Related selection functions:** Test current_by_station(), current_run_at(), and order_by_recency() to verify that the forecasts are selected and ordered consistently for individual stations and across multiple stations.
- **Report regression**: Run the existing tests in test_reports.py and test_trend.py, and compare the affected station and Trend report outputs with the saved baseline. Verify that the correct forecast is selected and that unrelated report formatting and existing behavior remain unchanged.
  These tests protect the forecast-selection rules identified in both tickets. We will first record the existing test results, then use the new regression tests to verify the intended behavior of any future implementation.

Rollback or Abort Plan

We will keep the proposed changes on a separate Git branch and preserve the original report outputs and test results. If a test fails, an older forecast is still selected, the report banner displays an incorrect timestamp, or unrelated report output changes unexpectedly, we will stop and investigate the cause. If the problem cannot be resolved safely, we will revert the change commit or abandon the branch and return to the last known-good version.
We will only consider the changes safe when the characterization tests pass, the existing reporting tests continue to pass, and the affected report outputs have been compared against the baseline.

4. Agent Appendix

How you will declare agent use:

- At the bottom of every page / deliverable a statement will be included stating that no AI was used or declaring the use of Agent and how it was used.
- If we are declaring that an Agent was used, we will be very clear and specific to highlight where and how we used it.
- Additionally, we will include what prompt the AI was given, what we used and what we did not.

In question 1:

AI was used to locate which folders and clients would be best used to support the genuine move behaviour close to data pattern. It was also used
Prompt:

1. don't edit the code, only use what's in the reporting folder, trace the clients within the reporting folder, find what resembles the pattern 'move behavior close to data'
2. create a diagram to model this behaviour that supports the judgment to keep forecast as it correctly represents the move behavior close to data
3. don't edit the code, only use what's in the reporting folder, trace the clients within the reporting folder, find what resembles the pattern 'eliminate navigation code' as a degenerate pattern (as an anti pattern)

Used:

1. The Core Pattern Target: Forecast as a Data Container . In Chapter 9 (Slides 4 & 16), a Data Container is defined as an object that provides access to data but possesses no behavior of its own (exhibiting public attributes/accessors and no domain methods). Consequently, clients must define the behavior themselves, which leads to duplicated logic across multiple clients.
   In db.py:

```
@dataclass(frozen=True)
class Forecast:
  stnid: str
  year: int
  run_at: str
  yield_t: float
  ndays: int
  model: str
```

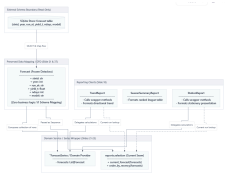
   Forecast is a passive, immutable data container with six fields and zero methods.
   2. Tracing the Clients of Forecast
   The direct clients that consume Forecast within the reporting package are:
   StationReport in station_report.py
   TrendReport in trend.py
   SeasonSummaryReport in season_summary.py
   3. Duplicated Client Behavior Resembling "Move Behavior Close to Data"
   Tracing how these clients interact with Forecast reveals the classic smell described in Slide 16 & Slide 18 (analogous to the Payroll vs. TelephoneGuide example on Employee):
   A. Trend Statistics Calculation (Near-Identical Duplication)
   Both StationReport and TrendReport extract raw fields (yield_t, year) from lists of Forecast objects to compute the arithmetic mean, min (worst), max (best), and net change:
   Client 1: StationReport._trend_block:

```
ys = [f.yield_t for f in forecasts]
mean = sum(ys) / len(ys)
lo = min(forecasts, key=lambda f: f.yield_t)
```

```
    hi = max(forecasts, key=lambda f: f.yield_t)
    first, last = forecasts[0], forecasts[-1]
    net = last.yield_t - first.yield_t
    lines = [
      "  Mean yield ..... %.3f t/ha over %d year(s)" % (mean, len(ys)),
      "  Best year ..... %d  (%.3f t/ha)" % (hi.year, hi.yield_t),
      "  Worst year ..... %d  (%.3f t/ha)" % (lo.year, lo.yield_t),
      "  Net change ..... %+.3f t/ha  (%d -> %d)" % (net, first.year, last.year),
    ]
```

Client 2: TrendReport._single_summary_block:

```
    ys = [f.yield_t for f in forecasts]
    mean = sum(ys) / len(ys)
    hi = max(forecasts, key=lambda f: f.yield_t)
    lo = min(forecasts, key=lambda f: f.yield_t)
    first, last = forecasts[0], forecasts[-1]
    net = last.yield_t - first.yield_t
    lines = [
      "  Mean yield ..... %.3f t/ha over %d year(s)" % (mean, len(ys)),
      "  Best year ..... %d  (%.3f t/ha)" % (hi.year, hi.yield_t),
      "  Worst year ..... %d  (%.3f t/ha)" % (lo.year, lo.yield_t),
      "  Net change ..... %+.3f t/ha  (%d -> %d)" % (net, first.year, last.year),
    ]
```

2.  use this diagram as a reference
3. Primary Evidence: The Document Aggregation Hierarchy in emitters.py
   Tracing the clients emit_csv and emit_json reveals an object-tree navigation chain through four distinct layers:
   doc (ReportDocument)⟶block (Block)⟶cs (ColumnSet)⟶cs.columns (List[Column])⟶c.title / c.key / c.unit / c.kind\text{doc (ReportDocument)} \longrightarrow \text{block (Block)} \longrightarrow \text{cs (ColumnSet)} \longrightarrow \text{cs.columns (List[Column])} \longrightarrow \text{c.title / c.key / c.unit / c.kind}doc (ReportDocument)⟶block (Block)⟶cs (ColumnSet)⟶cs.columns (List[Column])⟶c.title / c.key / c.unit / c.kind
   However, both emit_csv and emit_json bypass this existing method and navigate through cs.columns down to c.key manually.
   The Navigation Code Anti-Pattern (Violation of Law of Demeter):

The emitter functions know the entire internal object hierarchy of the report document down to individual field names of Column. If Column renames title to header_text or changes how keys are represented, emitters.py breaks, even though it should only care about formatting blocks of rows.

In question 2:
AI was used to help identify three decay signatures in Meridian and jq, and it gave examples in simpler terms which helped us understand and review the code.
Where AI was correct: it identified duplicated calculations in Meridians station and trend reports and repeated memory setup logic in jq. It checked the upload source files and Git history to support the file references and explanations.
Where AI needed corrections: The initial duplication comparison focused mainly on differences between the code examples without clearly explaining the human versus distinction.

In question 3, part 1:
In this question, AI was used to help review the proposed changes for tickets MRD-238 and MRD-250. I used to help identify the functions and their dependencies, as well as the ways we could test whether the changes would affect existing functionality. I checked the relevant source files and existing tests to verify the information I was given before including anything in the assignment.
Prompt given: "What functions and dependencies are affected by the proposed changes for MRD-238 and MRD-250, and how could we test whether the changes affect existing functionality?"
Where AI was correct: AI correctly identified that both tickets involve comparing forecast timestamps as text, which can cause an older forecast to be selected when timestamps use different formats. It also helped identify affected functions, including latest_forecast_for_station() in db.py and current_forecast() in selection.py. I verified this by checking the functions in the source files and searching the repository for where they are called.
Where AI was incorrect and how it was verified: AI's initial response did not identify all the functions affected by the proposed changes. It focused on latest_forecast_for_station() and current_forecast(), but missed other related functions, including latest_run_at(), which supplies the timestamp displayed in the report banner, and current_run_at(), which relies on the forecast selection logic. I found these by reviewing the source files and tracing the related functions myself. I then added them to the impact analysis and testing considerations.

In question 3, part 2 (Safe change):
In this question, AI was used to help review the proposed changes for MRD-238 and MRD-250, identify affected functions and suggest tests for checking existing functionality. Then these suggestions were used as a guide for review of the relevant source files.
Where AI was correct: AI correctly identifies the timestamp comparison tissue and relevant functions. These suggestions helped understand the potential impact of the proposed changes.
Where AI was incorrect and how it was verified: AI's initial tests were incomplete because they did not cover all relevant functions. These functions were added to the testing considerations to cover report timestamps and forecast recency.

Unless otherwise stated: no other AI help was used.